

MiniBench 代码讲解讲稿

agents / core / factory 模块梳理，以及 OpenAI-compatible、direct 与 prompt 设计说明

用途：晚上汇报时可以按本讲稿顺序讲。建议先讲整体运行链路，再进入每个文件的职责和关键函数。

项目路径	D:\AAALimoWork\CS\Seminar\MiniBench
讲解范围	src/minibench/agents, src/minibench/core, src/minibench/factory
核心主线	配置文件和 CLI 参数被 factory 装配成 agent 与 task family, dataset evaluation 调用 <code>Agent.generate</code> , core 负责 prompt、metrics、summary 与结果落盘。

1 总览：MiniBench 的分层

MiniBench 的代码结构比较清楚，可以按职责分成三层来讲：

- **agents**: 答题策略层。所有 agent 都实现 `generate(prompt, task) -> str`。
- **core**: 公共基础设施层。定义接口、通用 prompt、metrics、summary、run 文件写出。
- **factory**: 装配层。把 YAML/CLI 配置变成可运行的 agent、provider、task family 和 evaluation。

YAML/config

```
-> factory.config.validate_experiment_config
-> factory.experiments.get_task_family_spec
-> load_tasks
-> factory.agents.make_agent_from_config
-> dataset.evaluate_tasks(..., agent)
-> core.metrics / core.results / core.runs
-> runs/<run_name>/predictions.jsonl, results.json, summary.txt
```

汇报句：MiniBench 的设计重点是把模型调用、推理策略、任务数据集和结果统计解耦。这样新增 agent 或新增 task family 时，不需要重写整条评测链路。

2 agents 文件夹

agents 层不是具体模型实现，而是不同的答题策略。它们共同遵守 `Agent.generate` 接口：输入题目 `prompt` 和 `task`，返回模型或规则产生的字符串输出。

2.1 `__init__.py`

这个文件只是包出口，集中导出 `Agent`、`ChatClient`、`ReasoningConfig` 以及所有具体 agent 类。它不包含业务逻辑，作用是让外部可以统一从 `minibench.agents` 导入。

2.2 `simple.py`

这里放三个简单 agent，主要用于 baseline、调试和实验复现。

- **OracleAgent**: 直接读取 `task.correct_option`，返回标准 JSON。它是理论满分上界，用来验证评测流程。
- **NoisyAgent**: 也知道正确答案，但输出自然语言 `answer: A`，用于测试答案抽取器能否处理非严格 JSON。
- **PredictionFileAgent**: 从 JSONL 文件读取已有 `raw_output/raw_outputs`，按 `task.id` 顺序回放，适合复现实验结果。

2.3 `direct.py`

`DirectAgent` 是最简单的真实模型策略：只调用一次模型，但会用 `direct_prompt` 包装原始 `prompt`，并显式要求最终 JSON。

```
prompt -> direct_prompt(prompt)
        -> client.complete(..., FINAL_ANSWER_SYSTEM_PROMPT, json_mode=True)
        -> JSON answer
```

它适合作为最小 prompt-engineering baseline。

2.4 `cot.py`

`CoTAgent` 是 Chain-of-Thought，两阶段：第一阶段让模型自由推理，第二阶段把推理整理成严格 JSON。

```
reasoning = client.complete(cot_prompt(prompt), json_mode=False)
answer = client.complete(finalize_prompt(prompt, reasoning), json_mode=True)
```

它的设计思路是把“解题质量”和“输出格式稳定性”分开处理。

2.5 critic_refine.py

CriticRefineAgent 是先答、再批判、再修正。它对应 self-refinement 类思路。

- **draft:** 用 `direct_prompt` 先生成草稿。
- **critique:** 用 `critic_prompt` 让模型检查逻辑、约束和 schema。
- **refine:** 用 `refine_prompt` 根据批评修正，并输出最终 JSON。

2.6 plan_then_solve.py

PlanThenSolveAgent 是先规划再解题。它和 CoT 的区别是：CoT 是边想边解，Plan-Then-Solve 会先显式生成 plan，再按 plan 求解。

```
plan = client.complete(plan_prompt(prompt))
solution = client.complete(solve_with_plan_prompt(prompt, plan))
answer = client.complete(finalize_prompt(prompt, solution), json_mode=True)
```

2.7 self_consistency.py

SelfConsistencyAgent 是多次采样加多数投票。它先生成 `samples` 个 CoT 样本，然后尝试从每个样本里抽取 A/B/C/D。

- 如果有明确多数答案，直接返回多数票 JSON。
- 如果平票或抽取失败，则调用 `judge_prompt` 让模型在候选中选择最佳答案。

它降低了单次随机推理的偶然错误，尤其适合 `reasoning_temperature` 较高时。

2.8 tree_of_thought.py

TreeOfThoughtAgent 是简化版 Tree of Thought。它生成多个候选推理路径，然后统一交给 `judge_prompt` 选择最佳候选。

```
candidates = [candidate_prompt(prompt, i) for i in range(samples)]
answer = client.complete(judge_prompt(prompt, candidates), json_mode=True)
```

Self-consistency 更偏统计投票，Tree-of-Thought 更偏候选生成和评审。

agents 汇报总结：agents 文件夹实现的是不同推理范式，统一暴露 `generate` 接口。评测器不用知道内部是 `direct`、`CoT`、`ToT` 还是 `critic-refine`，只要调用 `generate` 即可。

3 core 文件夹

core 是跨任务复用的基础设施层。它不关心具体任务，而是提供所有 benchmark 都要用到的通用能力。

3.1 `__init__.py`

重新导出 `Agent`、`ChatClient`、`ReasoningConfig`，作为 core 包的公共 API 出口。

3.2 `agent.py`

这个文件定义 agent 系统的契约。

- **Agent:** 统一基类，要求子类实现 `generate(prompt, task) -> str`。
- **ChatClient:** Protocol 类型，只要对象有 `complete(...)` 方法，就可以作为模型客户端使用。
- **ReasoningConfig:** 统一管理 `samples`、`reasoning_temperature`、`final_temperature`、`max_reasoning_tokens`、`final_max_tokens`。

这里的关键设计是解耦:agent 策略只依赖 `ChatClient.complete`，不关心底层 provider 是 DeepSeek、Qwen、SiliconFlow 还是 generic API。

3.3 `prompts.py`

集中管理通用 prompt 模板。它定义三类 system prompt:

- `FINAL_ANSWER_SYSTEM_PROMPT`: 最终只输出一个 JSON object。
- `REASONING_SYSTEM_PROMPT`: 用于中间推理，禁止声称使用外部工具。
- `CRITIC_SYSTEM_PROMPT`: 用于审查草稿答案的逻辑和 schema。

主要 prompt builder 包括 `direct_prompt`、`cot_prompt`、`plan_prompt`、`solve_with_plan_prompt`、`candidate_prompt`、`judge_prompt`、`finalize_prompt`、`critic_prompt`、`refine_prompt`。

这里的 prompt 故意很短，更像通用 benchmark adapter。好处是减少任务特定 prompt 工程对 agent 架构比较的干扰；如果追求性能，可以进一步结构化。

3.4 `metrics.py`

`metrics.py` 负责统计任务耗时、模型调用耗时、LLM 调用次数和 token usage。它的核心方法是 `snapshot-delta`。

```
start = start_task_metrics(agent)
raw_output = agent.generate(prompt, task)
metrics = finish_task_metrics(agent, start)
```

- `start_task_metrics`: 记录 `perf_counter` 和模型指标快照。
- `finish_task_metrics`: 再次取快照, 用 `after - before` 得到本题增量。
- `model_metrics_snapshot`: 递归寻找 `agent` 或 `agent.client` 上的 `metrics_snapshot`。
- `summarize_metrics`: 汇总所有任务的 `total` 和 `average_per_task`。
- `extract_token_usage`: 递归扫描 `provider` 返回的 `usage`, 兼容 `prompt_tokens`、`completion_tokens`、`total_tokens` 以及其他 `*_tokens` 字段。

这个设计能兼容多步 `agent`, 因为一次 `generate` 内部可能调用 LLM 多次。

3.5 results.py

`results.py` 目前提供 `summarize_boolean_results`, 用于所有“每题成功或失败”的 benchmark。

- 整体统计 `total`、`success_count` 和 `rate`。
- 按 `tags` 分桶统计 `by_tag`。
- `success_attr`、`success_key`、`rate_key` 都由调用方传入, 因此可以复用在 `correct/accuracy` 或 `success/success_rate` 等不同命名场景。

3.6 runs.py

`runs.py` 负责实验目录和结果文件写出。

- `timestamped_run_dir`: 根据 `run_name`、`prefix` 或时间戳创建 `run` 目录, `exist_ok=False` 防止覆盖旧结果。
- `write_jsonl`: 把 `dataclass` 或 `dict` 写成 JSONL, 并使用 `ensure_ascii=False` 保留中文。
- `write_summary_artifacts`: 统一写 `predictions.jsonl`、`results.json`、`summary.txt`。

core 汇报总结: `core` 定义了 MiniBench 的公共运行协议, 包括 `agent` 接口、`prompt` 模板、`metrics` 统计、布尔结果汇总和实验产物落盘。

4 factory 文件夹

`factory` 是装配层, 负责把配置变成可运行实验。它本身不写具体任务逻辑, 而是把 `agents`、`providers`、`datasets` 和 `core` 连接起来。

4.1 `__init__.py`

作为 factory 包门面, 导出 `AGENT_NAMES`、`OpenAICompatibleAgent`、`ProviderConfig`、`make_agent`、`make_agent_from_config`、`resolve_provider`。它使用 `__getattr__` 做延迟导入, 减少初始化开销, 也降低循环导入风险。

4.2 `agents.py`

`agents.py` 根据字符串名字创建具体 agent。核心表是 `REASONING_AGENTS`: `direct`、`cot`、`self-consistency`、`tot`、`plan-then-solve`、`critic-refine`。

- 如果传 `predictions`, 优先创建 `PredictionFileAgent`。
- `name` 为 `oracle/noisy` 时创建对应 `baseline`。
- `name` 为 `openai-compatible` 时, 直接创建 `OpenAICompatibleAgent`。
- `name` 为 reasoning agent 时, 先创建 `OpenAICompatibleAgent` 作为 client, 再把 client 和 `ReasoningConfig` 包进具体策略 agent。

```
CoTAgent
-> OpenAICompatibleAgent.complete(...)

TreeOfThoughtAgent
-> OpenAICompatibleAgent.complete(...)
```

因此 `OpenAICompatibleAgent` 既可以单独当裸模型 `baseline`, 也可以作为其他 reasoning agent 的底层 client。

4.3 `providers.py`

`providers.py` 实现 OpenAI-compatible HTTP 客户端。所谓 OpenAI-compatible, 是指第三方服务虽然不是 OpenAI 官方, 但 API 形状兼容 OpenAI Chat Completions。

- 请求路径通常是 `/chat/completions`。
- 请求体包含 `model`、`messages`、`temperature`、`max_tokens`、`stream`。
- 鉴权使用 `Authorization: Bearer <api_key>`。
- 返回读取 `choices[0].message.content`。
- 可以用 `response_format: {type: json_object}` 请求 JSON mode。

`PROVIDERS` 内置 `deepseek`、`qwen`、`qwen-intl`、`qwen-us`、`siliconflow`。generic provider 则要求用户显式给 `model`、`base_url` 和 `api_key_env`。

`OpenAICompatibleAgent` 同时记录模型耗时、LLM 调用次数、usage 缺失次数和 token usage, 供 `core.metrics` 使用。

4.4 config.py

config.py 读取和校验实验 YAML。一个实验配置必须有四个 section: task、agent、provider、run。

- task.family 必须在 TASK_FAMILIES 中。
- task.task_ids 必须是字符串列表, null 会被规范化为空列表。
- task.limit 必须是正整数或 null。
- agent.name 必须在 AGENT_NAMES 中。
- provider.name 缺失时默认 generic。
- run.output_dir 缺失时默认 runs。

它的价值是让配置错误尽早暴露, 而不是跑到一半才失败。

4.5 experiments.py

experiments.py 是 factory 的主编排文件。它用 TaskFamilySpec 把不同任务家族抽象成统一接口。

```
TaskFamilySpec:
    default_path
    load_tasks
    evaluate_tasks
    summarize
    write_run
    system_prompt
```

目前 TASK_FAMILIES 支持 multiple_choice、xiangqi、one_stroke、mahjong、mahjong_riichi。每个 family 都注册自己的 load/evaluate/summarize/write_run 函数。

- run_family_experiment: 加载任务、筛选 task_ids、应用 limit、创建 agent、运行 evaluation、写出 run。
- _select_tasks: 按 id 过滤任务, 并对不存在的 id 报错。
- _evaluate: 为 xiangqi 和 mahjong_riichi 注入额外评测参数; 其他 family 直接 evaluate_tasks(tasks, agent)。

factory 汇报总结: factory 把配置文件中的 task、agent、provider 和 run 四块组合成一次可执行实验。新增 task family 时, 只要提供 load/evaluate/summarize/write_run 并注册到 TASK_FAMILIES, 就能复用整条运行链路。

5 OpenAI-compatible 与 direct 的区别

这是很容易被问到的问题，因为 openai-compatible 和 direct 都出现在 agent 名称列表里，而且都只调用一次模型。区别在于定位不同。

openai-compatible	裸模型调用器，也是底层 provider client。它基本把 dataset 生成的 prompt 原样发给模型，依赖 provider/default system prompt 约束输出。
direct	一个最小直接答题策略。它会用 <code>direct_prompt</code> 包装原始 prompt，并显式要求最终 JSON。

```
openai-compatible:
  prompt -> OpenAICompatibleAgent.complete(prompt) -> raw output

direct:
  prompt -> direct_prompt(prompt)
    -> OpenAICompatibleAgent.complete(
      system_prompt=FINAL_ANSWER_SYSTEM_PROMPT,
      json_mode=True
    )
    -> JSON answer
```

项目	openai-compatible	direct
定位	provider/base model agent	最小 prompt-engineering 策略
是否包装 prompt	基本不包装	使用 <code>direct_prompt</code> 包装
JSON mode	取决于 provider 配置	调用时显式 <code>json_mode=True</code>
是否可做其他 agent 的底层 client	是	通常不是
调用次数	1 次	1 次

汇报句：openai-compatible 测的是模型在原始任务 prompt 下的裸表现；direct 测的是一个最小直接答题策略。它们都不是多步推理，但 direct 更像 agent 架构里的 baseline 策略，openai-compatible 更像底层模型适配器顺便暴露成 baseline。

6 prompts.py 为什么简单，以及如何增强

当前 `prompts.py` 是最小可用版，适合框架验证和公平比较。它的优点是 token 成本低、通用性强、不容易把策略评测变成任务特定 prompt 工程。

但如果目标从框架验证转向性能优化，可以参考开源项目把 prompt 写得更结构化。

6.1 开源项目常见写法

- **few-shot 示例:** Tree-of-Thoughts 官方 repo 的 Game of 24 prompt 给了多个完整示例, 展示每一步计算和剩余数字。
- **生成和评估分离:** ToT 通常会有 propose_prompt、value_prompt、vote_prompt, 不同阶段有不同 prompt。
- **judge rubric:** 裁判 prompt 不只是“选最好”, 而是明确检查 correctness、constraint following、schema compliance、confidence。
- **任务特定格式:** crossword、game24 等任务会把规则、状态表示、输出格式和 confidence 标签写得非常细。
- **Auto-CoT:** 通过自动生成和筛选 demonstration, 减少人工写 few-shot CoT 的成本。
- **Self-Refine:** initial output -> feedback -> refinement, critic prompt 和 refine prompt 通常带检查清单。

6.2 可以增强的方向

- **direct_prompt:** 增加读题、比较选项、排除错误项、最终 JSON 的协议。
- **cot_prompt:** 增加分段要求, 如 Analysis 和 Answer, 但最终仍由 finalize_prompt 转成 JSON。
- **candidate_prompt:** 要求不同 candidate 尝试不同解法, 避免重复。
- **judge_prompt:** 增加评分 rubric, 并要求最后只输出 JSON。
- **critic_prompt:** 增加检查清单, 明确检查逻辑、schema、选项范围和约束。
- **task family prompt:** 对象棋、麻将、一笔画等任务使用更任务特定的 reasoning prompt。

prompt 汇报总结: 现在的 prompt 简单是为了通用和公平; 复杂 prompt 可以提高性能, 但会引入更多 prompt engineering。后续优化可以在保持最终 JSON schema 稳定的前提下, 把中间推理、候选生成和裁判阶段写得更结构化。

7 收尾话术

MiniBench 的核心设计是将评测流程拆成可替换的模块: agent 负责答题策略, core 负责通用接口和统计落盘, factory 负责从配置装配实验。OpenAI-compatible 是底层模型适配器, direct/CoT/ToT/self-consistency 等是不同推理策略。当前 prompt 保持简洁, 适合作为通用 benchmark 框架; 未来如果追求更高效果, 可以参考 ToT、Auto-CoT、Self-Refine 的写法, 引入 few-shot、rubric、候选评审和任务特定 prompt。

7.1 参考来源

- DeepSeek API Docs: <https://api-docs.deepseek.com/>
- Tree of Thoughts official repo prompts: <https://github.com/princeton-nlp/tree-of-thought-llm>
- Auto-CoT repo: <https://github.com/amazon-science/auto-cot>
- Self-Consistency paper: <https://arxiv.org/abs/2203.11171>
- Self-Refine paper: <https://arxiv.org/abs/2303.17651>